

最优子结构

动态规划 · 8题 · C++ & Python 双语对照

NQ146 AcWing 2 AcWing 2

AcWing 2

[输入](#) 见原题[输出](#) 见原题[来源](#) AcWing 2**C++**

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 1007;
// 2维的解法
int n, m;
int f[N][N];
int v[N], w[N];

int main()
{
    cin >> n >> m; // 读入 N, V
    // 读入 v, w
    for (int i = 1; i <= n; i++) cin >> v[i] >> w[i];
    // 朴素的二维dp
    for (int i = 1; i <= n; i++)
        for (int j = 0; j <= m; j++)
        {
            f[i][j] = f[i-1][j];
            if (j >= v[i]) // 剩余体积大于第i项的体积
                f[i][j] = max(f[i][j], f[i-1][j-v[i]] + w[i]);
        }
    // 根据定义，从前n个物品中，选取总体积不超过m的选法集合f[n][m]就是答案
    cout << f[n][m] << endl;
}
```

Python

```
# AcWing 2
# Python solution
```

NQ147 AcWing 3 AcWing 3

AcWing 3

[输入](#) 见原题

输出

见原题

来源

AcWing 3

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 1007;

int n,m;
int f[N][N];
int v[N],w[N];

int main()
{
    cin >>n >>m;
    for (int i = 1; i <= n; i++) cin>>v[i]>>w[i];

    //完全背包
    // 1.f[i][j] = f[i-1][j];(不选物品i)
    // 2.f[i][j]= max(f[i-1][j], f[i][j-v]+w)
    (选若干个物品i)
    for (int i = 1; i <= n; i++)
        for (int j=0; j <= m; j++)
        {
            f[i][j]=f[i-1][j];
            if (j >= v[i])
                f[i][j] = max(f[i-1][j], f[i][j-v[i]]
+w[i]);
        }

    cout<<f[n][m]<<endl;
    return 0;
}
```

Python

```
# AcWing 3
# Python solution
```

NQ148 数字三角形 AcWing 898

给定一个如下图所示的数字三角形，从顶部出发，在每一结点可以选择移动至其左下方的结点或移动至其右下方的结点，一直走到底层，要求找出一条路径，使路径上的数字的和最大。7 3 8 8 1 0 2 7 4 4 4 5 2 6 5

输入

第一行包含整数n，表示数字三角形的层数。接下来n行，每行包含若干整数，其中第 i 行表示数字三角形第 i 层包含的整数。

输出

输出一个整数，表示最大的路径数字和。

来源

AcWing 898

输入

```
5
7
3 8
```

输出

```
30
```

```
8 1 0
2 7 4 4
4 5 2 6 5
```

提示 原题链接

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 507;

int n;
int w[N][N], f[N][N]; //三角矩阵依旧用2维存储

int main()
{
    //读入n, w
    cin >> n;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= i; j++)
            cin >> w[i][j];

    //初始化最底层的f[n][j]
    for (int i = 1; i <= n; i++) f[n][i] = w[n][i];

    //用dp从底层往上计算每个f[i][j]的值
    for (int i = n-1; i; i--)
        for (int j = 1; j < n; j++)
            f[i][j] = max(f[i+1][j]+w[i][j], f[i+1][j+1] + w[i][j]);

    cout << f[1][1] << endl;
    return 0;
}
```

Python

```
# AcWing 898
# Python solution
```

NQ149 最长上升子序列 AcWing 895

给定一个长度为N的数列，求数值严格单调递增的子序列的长度最长是多少。

输入 第一行包含整数N。第二行包含N个整数，表示完整序列。

输出 输出一个整数，表示最大长度。

来源 AcWing 895

输入

```
7
3 1 2 1 8 5 6
```

输出

```
4
```

提示 原题链接

C++

```
#include <iostream>
#include <algorithm>
#include <cstring>
using namespace std;
// f[i]定义为以s[i]结尾的最长的子序列
// 集合: 所有以第i个数结尾的上升子序列集合
// 属性: Max 上升子序列长度的最大值
const int N = 10007;
int n;
int a[N], f[N];
//状态计算
int main()
{
    cin >> n;
    for (int i = 1; i <= n; i++)
        cin >> a[i]; //从1开始初始化

    for (int i = 1; i <= n; i++)
    {
        f[i] = 1; // a[i]只有一个元素, 所以f[i]=1;
        for (int j = 1; j < i; j++) //求i之前的f[j]
            if (a[j] < a[i])
                f[i] = max(f[i], f[j] + 1);
    }

    int res = 1;
    for (int i = 1; i <= n; i++)
        res = max(res, f[i]);
    cout << res << endl;
    return 0;
}
```

Python

```
# AcWing 895
# Python solution
```

NQ150 最长公共子序列 AcWing 897

给定两个长度分别为N和M的字符串A和B，求既是A的子序列又是B的子序列的字符串长度最长是多少。数据范围 $1 \leq N, M \leq 1000$

输入

第一行包含两个整数N和M。第二行包含一个长度为N的字符串，表示字符串A。第三行包含一个长度为M的字符串，表示字符串B。字符串均由小写字母构成。

输出

输出一个整数，表示最大长度。

来源

AcWing 897

输入

```
4 5
acbd
abedc
```

输出

```
3
```

提示 原题链接 Y总讲解

C++

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
    {
        f[i][j] = max(f[i-1][j], f[i][j-1]);
        if (a[i] == b[j]) f[i][j] = max(f[i][j], f[i-1][j-1] + 1);
    }

cout << f[n][m] << endl;
return 0;
```

Python

```
# AcWing 897
# Python solution
```

NQ151 石子合并 AcWing 282

设有N堆石子排成一排，其编号为1, 2, 3, ..., N。每堆石子有一定的质量，可以用一个整数来描述，现在要将这N堆石子合并成一堆。每次只能合并相邻的两堆，合并的代价为这两堆石子的质量之和，合并后与这两堆石子相邻的石子将和新堆相邻，合并时由于选择的顺序不同，合并的总代价也不相同。例如有4堆石子分别为1 3 5 2，我们可以先合并1、2堆，代价为4，得到4 5 2，又合并 1, 2堆，代价为9，得到9 2，再合并得到11，总代价为4+9+11=24；如果第二步是先合并2, 3堆，则代价为7，得到4 7，最后一次合并代价为11，总代价为4+7+11=22。问题是：找出一种合理的方法，使总的代价最小，输出最小代价。数据范围 $1 \leq N \leq 300$

输入

第一行一个数N表示石子的堆数N。第二行N个数，表示每堆石子的质量(均不超过1000)。

输出

输出一个整数，表示最小代价。

来源

AcWing 282

输入

4
1 3 5 2

输出

22

提示 原题链接 ACWing讲解

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 307;

int n;
int s[N]; //前缀和
int f[N][N]; //所有将[i,j]合并成一堆的方案数

int main()
{
    cin>>n;
    for (int i = 1; i <= n; i++) cin>>s[i], s[i] += s[i-1]; //计算前缀和

    //区间DP, 先枚举长度, 后枚举端点
    for (int len = 2; len <= n; len++)
        for (int i = 1; i+len-1 <= n; i++) //区间左端点
        {
            //区间右端点
            int j = i + len - 1;
            f[i][j] = 1e9; //初始化为无穷大
            for (int k = i; k < j; k++) //区间dp, 枚举分隔点
                f[i][j] = min(f[i][j], f[i][k]+f[k+1][j]+s[j]-s[i-1]);
        }

    cout<<f[1][n]<<endl;
    return 0;
}

```

Python

```

# AcWing 282
# Python solution

```

NQ152 最短编辑距离 AcWing 902

给定两个字符串A和B，现在要将A经过若干操作变为B，可进行的操作有：删除—将字符串A中的某个字符删除。插入—在字符串A的某个位置插入某个字符。替换—将字符串A中的某个字符替换为另一个字符。现在请你求出，将A变为B至少需要进行多少次操作。数据范围 $1 \leq n, m \leq 1000$

输入

第一行包含整数n，表示字符串A的长度。第二行包含一个长度为n的字符串A。第三行包含整数m，表示字符串B的长度。第四行包含一个长度为m的字符串B。字符串中均只包含大写字母。

输出

输出一个整数，表示最少操作次数。

来源

AcWing 902

输入

```

10
AGTCTGACGC
11
AGTAAGTAGGC

```

输出

```

4

```

提示 原题链接

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;
const int N = 1007;

int n,m;
char a[N],b[N];
int f[N][N]; //所有將a[i]變為b[i]的操作步驟

int main()
{
    cin>>n>>a+1; //下標從1開始
    cin>>m>>b+1;

    //初始化
    for (int i = 0; i < N; i ++ ) f[i][0] = i; //把a[i]變為空串
    for (int i = 0; i < N; i ++ ) f[0][i] = i; //把空串變為b[i]

    // DP
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
        {
            f[i][j] = min(f[i-1][j] + 1, f[i][j-1] + 1);
            if (a[i]==b[j]) f[i][j] = min(f[i][j], f[i-1][j-1]);
            else f[i][j] = min(f[i][j], f[i-1][j-1] + 1);
        }

    cout<<f[n][m]<<endl;

    return 0;
}
```

Python

```
# AcWing 902
# Python solution
```

NQ153 滑雪 AcWing 901

给定一个R行C列的矩阵，表示一个矩形网格滑雪场。矩阵中第*i*行第*j*列的点表示滑雪场的第*i*行第*j*列区域的高度。一个人从滑雪场中的某个区域内出发，每次可以向上下左右任意一个方向滑动一个单位距离。当然，一个人能够滑动到某相邻区域的前提是该区域的高度低于自己目前所在区域的高度。下面给出一个矩阵作为例子：1 2 3 4 5 16 17 18 19 6 15 24 25 20 7 14 23 22 21 8 13 12 11 10 9 在给定矩阵中，一条可行的滑行轨迹为24-17-2-1。在给定矩阵中，最长的滑行轨迹为25-24-23-...-3-2-1，沿途共经过25个区域。现在给定你一个二维矩阵表示滑雪场各区域的高度，请你找出在该滑雪场中能够完成的最长滑雪轨迹，并输出其长度(可经过最大区域数)。

输入

第一行包含两个整数R和C。接下来R行，每行包含C个整数，表示完整的二维矩阵。

输出

输出一个整数，表示可完成的最长滑雪长度。

来源

AcWing 901

输入

```
5 5
1 2 3 4 5
16 17 18 19 6
15 24 25 20 7
14 23 22 21 8
13 12 11 10 9
```

输出

```
25
```

提示 [原题链接](#)

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 307;

int g[N][N]; // 地图高度
int f[N][N]; // 从[i,j]开始滑的最长的轨迹

int n, m; // 宽高

int dx[4] = {-1, 0, 1, 0}, dy[4] = {0, 1, 0, -1};

int dp(int x, int y)
{
    // 记忆化搜索
    if (f[x][y] != -1) return f[x][y];
    // 否则, 从x,y开始滑
    f[x][y] = 1; // 当前位置至少是1格
    // 分上下左右四个方向滑雪
    for (int i = 0; i < 4; i++)
    {
        int a = x + dx[i], b = y + dy[i];
        if (a < 1 || a > n || b < 1 || b > m) continue;
        // 越界
        if (g[a][b] >= g[x][y]) continue; // 不能走
        // 四个方向深搜, 更新f[a][b], 取最大值
        f[x][y] = max(f[x][y], dp(a, b) + 1);
    }

    return f[x][y]; // 返回f[x][y]
}

int main()
{
    cin >> n >> m;
    // 读入高度图
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            cin >> g[i][j];

    memset(f, -1, sizeof f); // 初始化为-1, 表示该点未被访问

    // 把每个点作为起点深搜
    int res = 0;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            res = max(res, dp(i, j));

    cout << res << endl;
    return 0;
}

```

Python

```

# AcWing 901
# Python solution

```